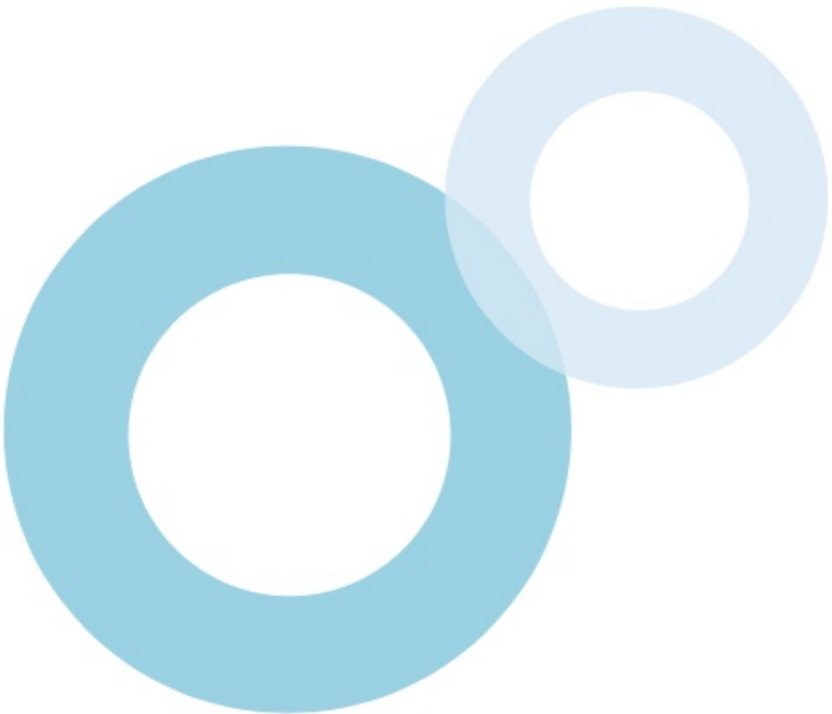
Build Solution

Given a use case, determine error handling for different integration options.

Table of Contents

After studying this topic, you should be able to:

- Identify the integration options available for a specific integration pattern.
- Determine how to handle errors based in the integration option that is being used.
- Describe the error handling strategy to implement in a given use case.



Introduction

Salesforce provides several tools for integrating the platform with other systems. A specific set of solution options are available based on the integration pattern that is implemented. When using these tools such as **External Services**, **Outbound Messaging**, **Salesforce APIs**, **Platform Events**, **Change Data Capture**, and **Salesforce Connect** in an integration solution, it is essential to be able to identify when errors can occur, which system should perform custom error handling, and even more so how to handle the errors to maintain a stable and reliable application.

In this topic, **error handling** mechanisms related to these **integration options** will be covered as well as **use cases** that illustrate how they're implemented in real-world scenarios.

Overview

Streaming API

In this solution option, the client can handle errors accordingly based on error response codes received in the streaming channel.

Platform Events

Error handling is implemented in the remote system as the event messages are handed off for processing.

Change Data Capture

The remote system, being the consumer in this integration solution, manages the error handling.

Salesforce Connect

In data virtualization, the caller manages the error handling when errors are encountered.

SOAP / REST / Bulk APIs

Error handling should be considered as part of the overall solution. The application initiating the callout performs the error handling.

Outbound Messaging

Delivered in a fire-and-forget manner, the remote system using this outbound call should perform error handling.

External Services

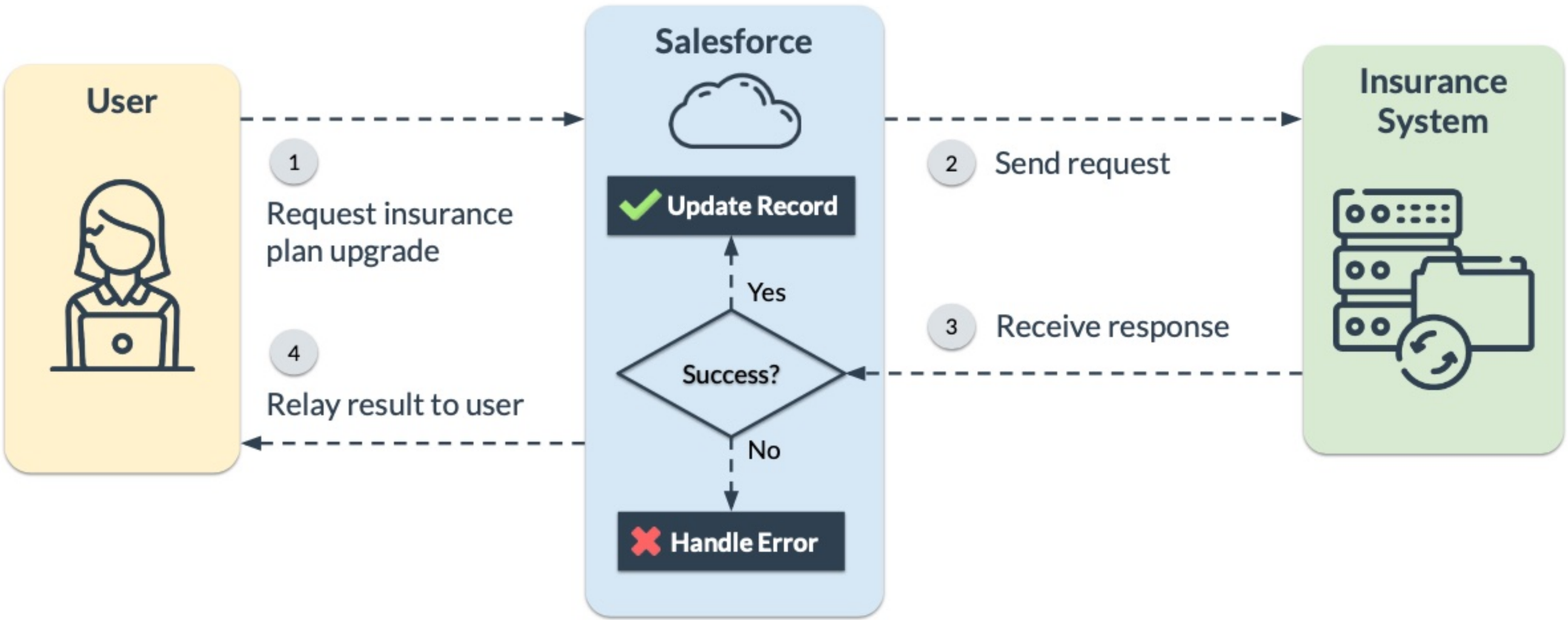
The remote invocation is initiated using Flow Builder. Error handling using this solution is managed by the caller.

Remote Process Invocation - Request and Reply

Table of Contents

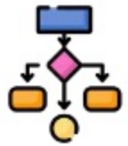
Error Handling

In the *Remote Process Invocation - Request and Reply* pattern, the **caller** is expected to **handle errors**. No changes should be committed **unless a successful response** is received.



Integration Solutions

In the **Remote Process Invocation - Request and Reply** pattern, the following Salesforce tools can be used as **integration solutions**.



EXTERNAL SERVICES

A flow is used to communicate with an external system using REST API in a declarative manner.

LIGHTNING COMPONENT

An Aura or Lightning web component is used to perform SOAP or REST callouts to an external system.

VISUALFORCE PAGE

A custom Visualforce page is used to submit an HTTP request to an external system.

APEX TRIGGER

An Apex trigger is used to perform asynchronous SOAP or HTTP callouts to an external system.

BATCH APEX

An Apex job can submit requests and process responses from an external system using SOAP or HTTP callouts.

External Services

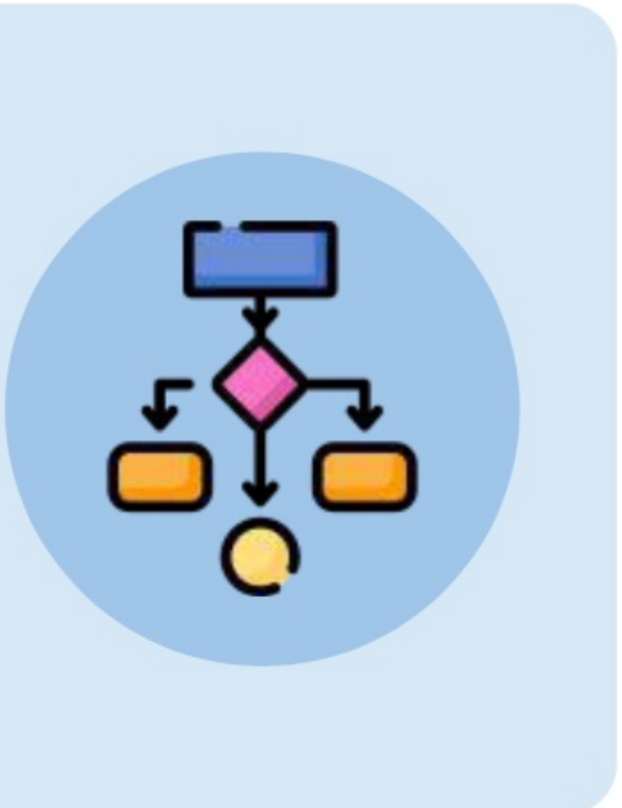
To communicate with the remote system, external services use a **REST API schema definition** which contains **response codes** that can be used for determining errors.

FLOW BUILDER

A flow can be used to **perform actions** specified by the schema on the remote system. The status of an operation can be determined by the **status code** included in the returned response.

DECISION ELEMENT

Based on the **status code** in the response, the Decision element can be used to **execute the next path**. For example, **update a record** if the operation was successful, or **display an error screen** if otherwise.



External Services

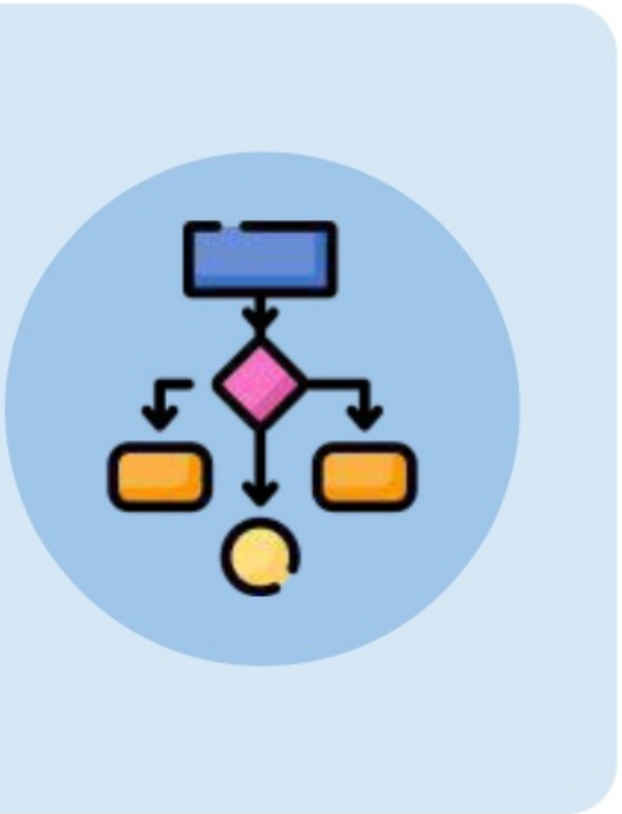
To communicate with the remote system, external services use a **REST API schema definition** which contains **response codes** that can be used for determining errors.

FAULT PATH

In case an unexpected error was encountered while executing an action, the **Fault Path** can be added to the **Action Element** to handle the error gracefully and execute an alternative path.

FAILED FLOW

If an error is unhandled in Flow Builder, an **email notification** is triggered that contains **detailed information** of the error and a link to the **failed flow interview** for debugging.



Lightning Components

When a Lightning component **sends** a request to a **server-side controller**, the action may cause a **server-side error**. Server-side errors should be caught and handled accordingly in **Apex**.

❖ TRY-CATCH BLOCK

Apex code or logic that **potentially cause errors** should be wrapped in try-catch blocks in order to **handle errors** and **recover** gracefully.

❖ EXCEPTION TYPES

Using **multiple** try-catch statements is recommended to catch and identify the specific **type of exception** that is encountered.

❖ THROW ERROR

The **AuraHandledException** should be thrown from the **catch block** to provide a **friendly error message** to the Lightning component.

Lightning Components

When a Lightning component **sends** a request to a **server-side controller**, the action may cause a **server-side error**. Server-side errors should be caught and handled accordingly in **Apex**.

❖ CALLOUT EXCEPTION

Whenever **Apex** encounters an error when performing a callout to an external system, the **CalloutException** exception type is thrown.

❖ ASYNC EXCEPTION

Whenever **Apex** encounters an error in an asynchronous operation such as failing to enqueue a job, the **AsyncException** exception type is thrown.

Lightning Components

The **client-side controller** of an **Aura component** may encounter errors when **sending requests** or **processing responses** from the server.

❖ CALLBACK FUNCTION

In **Aura components**, the callback function of an asynchronous operation contains an object which stores the response state. This data can be used to determine whether an **error is encountered** in the request and then perform the necessary handling.

❖ RESPONSE STATE

When an Apex controller throws an **AuraHandledException**, the state of the response that is captured in the callback function is set as ***ERROR***.

Lightning Components

When **wiring Apex methods to a function** in **Lightning web components**, the response of the server-side request is stored in either the **error** or **data** property of a response object.

❖ ERROR PROPERTY

If an error was encountered in the request, **error details** are stored in the **error** property. So, if this **property contains a value**, then an error has occurred and should be handled accordingly in the **wired method**.

❖ DATA PROPERTY

If **no error** occurred during the request, but data is received in an **unexpected format**, then a client-side error can potentially be thrown when processing values in the **data** property. In this case, the logic for processing the data should be wrapped in a client-side **try-catch block**.

Lightning Components

When using **imperative calls or promises** in **Lightning web components**, there are two options for handling errors .

❖ CATCH METHOD

The **promise object** contains a catch method that will be invoked when a server-side error is encountered in the request. It provides a parameter containing **error details** which can be used in the error handling.

❖ ASYNC/AWAIT

Alternatively, the async/await pattern can be used to execute the asynchronous call in a **try-catch block**. If an error is encountered during the request, it will be caught and should be handled in the **catch block**.

Visualforce Page

Visualforce page contain actions that invoke **custom server-side logic** that may encounter errors and require proper handling mechanisms on the **server-side** and **client-side**.

APEX CONTROLLER

Like any other Apex code, custom logic in the Visualforce controller that may encounter errors should be wrapped in **try-catch** blocks.

DISPLAY ERRORS

To display errors on the Visualforce page, **ApexPages.addmessage()** is called and **<apex:pageMessages>** should added to the page markup.

CONTINUATION

The **callback method** of a continuation contains the **response state**. Continuation-specific errors are specified in **>= 2000** status codes.

Apex Triggers

Apex triggers are allowed to **perform callouts** but only asynchronously by using **future methods**.

❖ TRY-CATCH BLOCK

In the **future method** that will be called from the **Apex trigger**, a try-catch block can be used to handle any errors.

❖ ASYNCHRONOUS

Since a future method is performed in **another transaction**, the request response cannot be relayed in the **original transaction**.

❖ ERROR REPORTING

When an error is caught in a **future method**, the common approach is to send an **email notification** containing the error details.

Batch Apex

Batch Apex can **perform callouts** by implementing the **Database.AllowsCallouts** interface.

ASYNCHRONOUS

Batch Apex is asynchronous so request responses cannot be relayed in the **original transaction** where the request was **initiated from**.

ERROR EMAIL

When an **unhandled error** is encountered, an email containing details of the error in the batch process is triggered.

PLATFORM EVENT

A platform event can be fired when an **unhandled error** is encountered by implementing the **Database.RaisesPlatformEvents** interface.

Batch Apex

Batch Apex can **perform callouts** by implementing the **Database.AllowsCallouts** interface.

❖ STORE ERROR

Errors can be captured using a **try-catch block**, and the error details including the **Job Id** can be stored in a custom object. Moreover, an **email alert** can be configured to automatically be sent on record creation.

❖ JOB STATUS

The **status** of a queued job can be monitored programmatically by querying **AsyncApexJob** using the **Job Id** via SOQL, or by navigating to '**Apex Jobs**' in **Setup**.

Remote Process Invocation - Fire and Forget

Table of Contents

Error Handling

In the **Remote Process Invocation - Fire and Forget** pattern, the **error handling** mechanism depends on the **integration solution** used which may be one or more of the following.

PLATFORM EVENTS

When a record is inserted or updated, Process Builder, Flow Builder, or Apex triggers/classes can publish a platform event to invoke the remote process.

OUTBOUND MESSAGING

When a record is inserted or updated, an outbound message can be sent to a remote SOAP endpoint using workflow rules or Flow Builder.

APEX CALLOUTS

An asynchronous SOAP/HTTP callout can be initiated from a Lightning component, Visualforce page, Apex trigger/class, or batch Apex to invoke the remote process.

Platform Events

The following describes the **error handling** mechanism when working with platform events in this integration pattern.

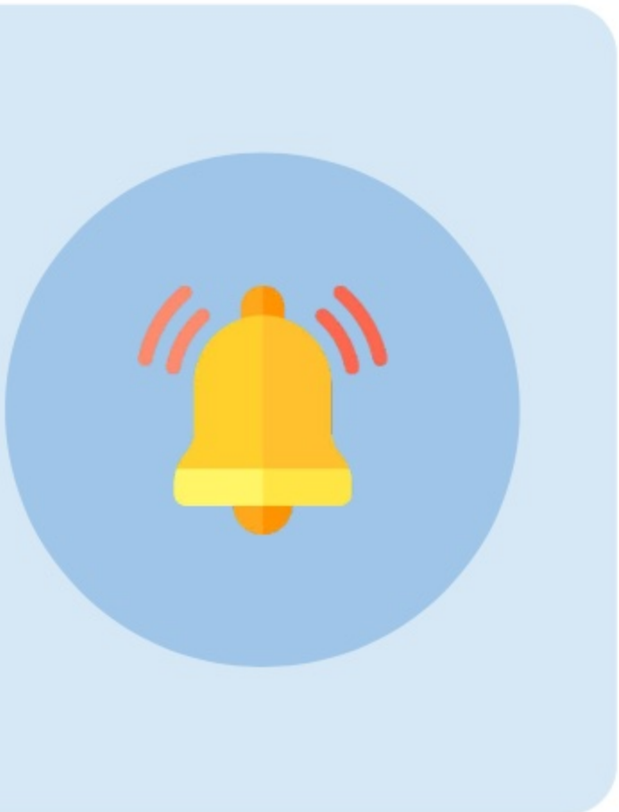
ERROR HANDLING

The message queuing, processing, and error handling must be handled entirely in the **remote system**. Unlike normal database operations, published platform events **can't be rolled back** within a transaction.

RECOVERY

Platform events generate a **replay ID** which can be used to replay the stream for a specific event that was published. In addition:

- Platform event messages are stored for **72 hours**.
- **Past event messages** can be retrieved when using **CometD** to subscribe to the channel.



Outbound Messaging

The following describes the **error handling** mechanism when working with outbound messaging in this integration pattern.

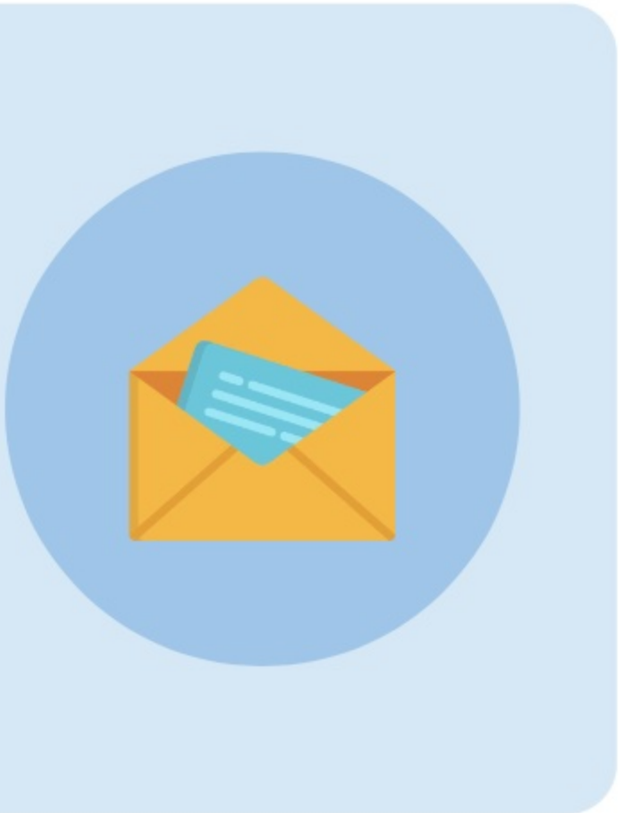
ERROR HANDLING

Custom error handling must be handled in the **remote system**. Should the remote system **fail to respond** with a positive acknowledgement within the timeout period, which is **20 seconds**, Salesforce automatically **retries the operation** for up to 24 hours.

RECOVERY

The **retry operation interval** increases exponentially overtime from **15-second** to **60-minute** intervals. Also, note the following:

- The **timeout period** can be extended to **7 days** by Salesforce support.
- Failed messages are placed in a **queue** which can be **retried manually**.



Apex Callouts

The following describes the **error handling** mechanism when working with Apex callouts in this integration pattern.

ERROR HANDLING

The caller handles exceptions only in the **initial invocation** of the remote service. For instance, a request timeout can be thrown when no positive acknowledgement is received. When the asynchronous **remote process has started**, any errors should be handled by the **remote system**.

RECOVERY

When an **error is encountered** in the initial invocation, a **custom retry mechanism** can be implemented if required. For example, a Lightning component can provide a button that allows a user to **retry the operation** if the remote system failed to respond previously.

Batch Data Synchronization

Table of Contents

Error Handling

In the **Batch Data Synchronization** pattern, how the **error handling** is implemented depends on the **error location**, which can be one or more of the following.

CHANGE DATA CAPTURE

An error is encountered when reading Salesforce data using Change Data Capture.

THIRD-PARTY ETL SYSTEM

An error is encountered when reading Salesforce data using a 3rd party ETL system.

WRITE TO SALESFORCE

An error is encountered from a write operation performed in the Salesforce database.

EXTERNAL MASTER SYSTEM

An error is encountered when an operation is performed in the master external system.

Change Data Capture

The following describes **error handling** considerations when an error is encountered upon **reading from Salesforce** using Change Data Capture.

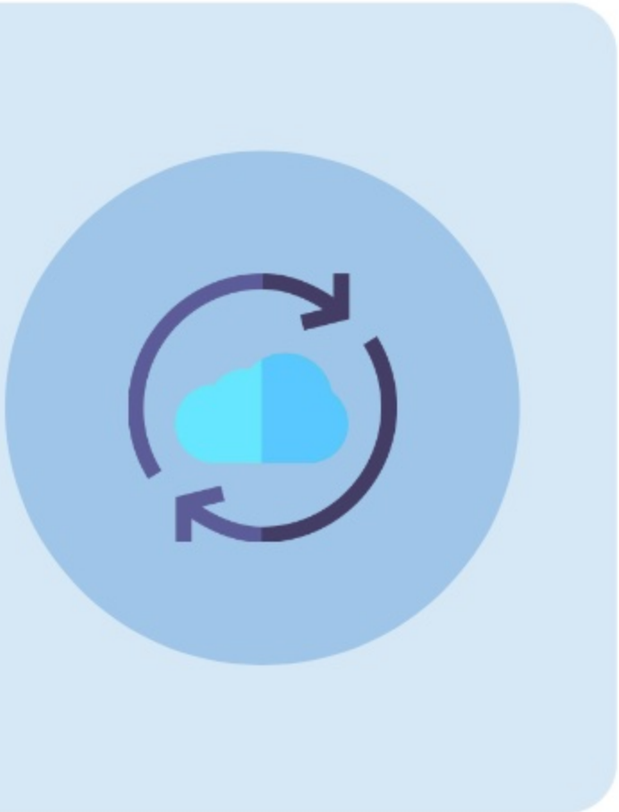
ERROR HANDLING

Since the **Change Data Capture client** is the remote system, the message queueing, processing, and error handling should be **assumed by the remote system**. Change Data Capture events are a **special type of platform events** and cannot be rolled back within a transaction.

RECOVERY

Change Data Capture events generate a **replay ID** which can be used to replay the stream for a specific event that was published. In addition:

- Change Data Capture event messages are stored for **72 hours**.
- **Past event messages** can be retrieved when using **CometD** to subscribe to the channel.



Change Data Capture

The following describes **error handling** considerations when an error is encountered upon **reading from Salesforce** using Change Data Capture.

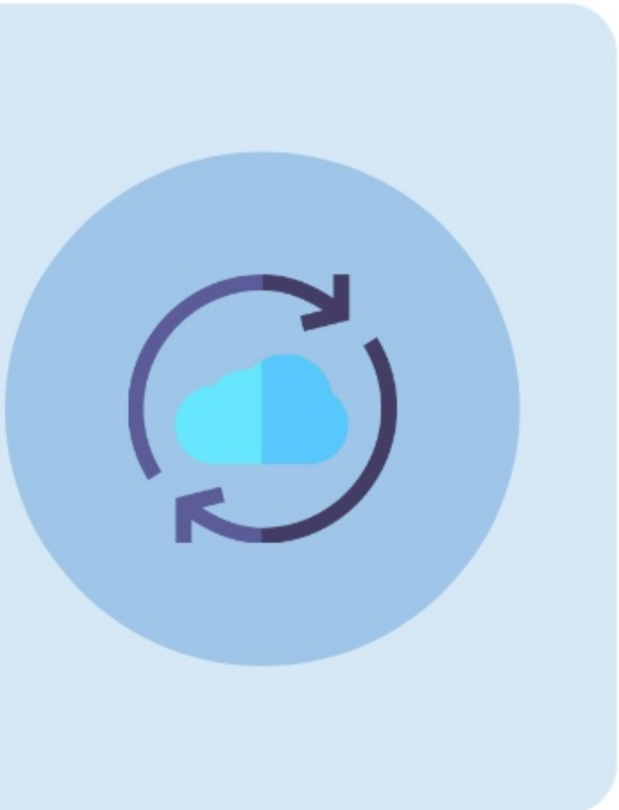
GAP EVENTS

When Salesforce is **unable to send change events**, a gap event is sent to **notify subscribers**. A gap event may be sent when:

- A change event size exceeds the **maximum allowed size** (1MB)
- Certain field types of an object are **currently being converted**
- An **internal error** has occurred in the Salesforce platform
- Unrelated database operation **is in progress** (i.e. archiving Activities)

HANDLE GAP EVENT

A gap event includes the IDs of the changed records. The **record IDs** can be used to retrieve the record from Salesforce using, for example, the **retrieve()** call of the Salesforce SOAP API.



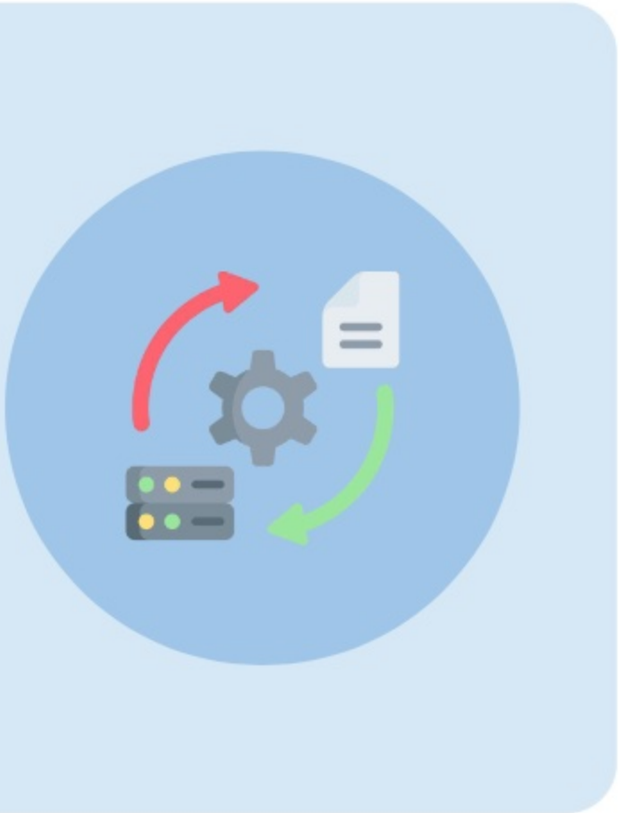
Third-Party ETL System

The following describes **error handling** considerations when an error is encountered upon **reading from Salesforce** using a 3rd-party ETL system.

ERROR HANDLING

If an error is encountered when **reading from Salesforce**, the remote system should execute a **retry mechanism**. If repeated failures are encountered, the following should be performed in the **remote system**:

- Log the error
- Retry the read operation
- Terminate the operation if unsuccessful
- Send a notification



Third-Party ETL System

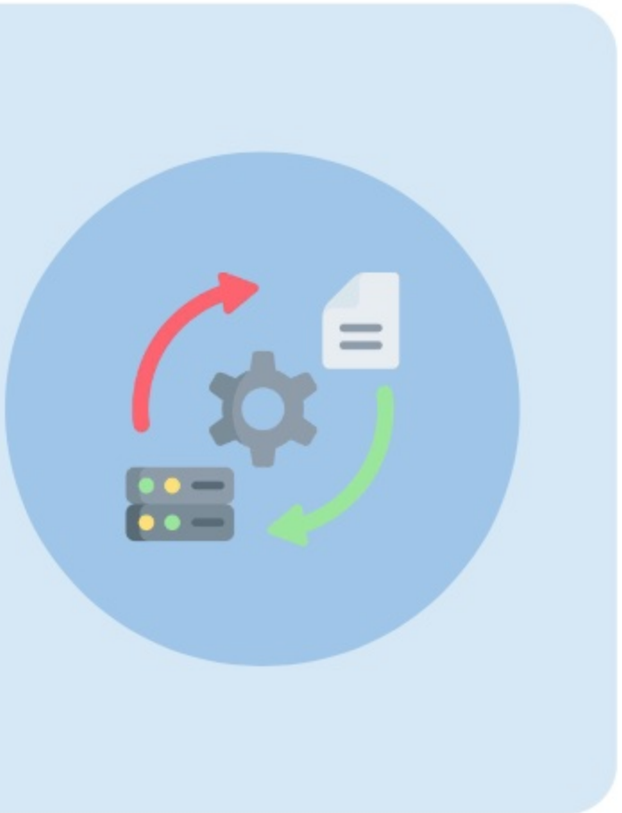
The following describes **error handling** considerations when an error is encountered upon **reading from Salesforce** using a 3rd-party ETL system.

RECOVERY

To recover from a **failed read operation**, the ETL process should be **restarted**.

FAILED RECORDS

If there are failed records even though the **read operation was successful**, an **immediate restart** of the operation should fix the issue. On the other hand, a **delayed restart** may be beneficial as it allows time for the data to correct itself in Salesforce which may be causing the errors.



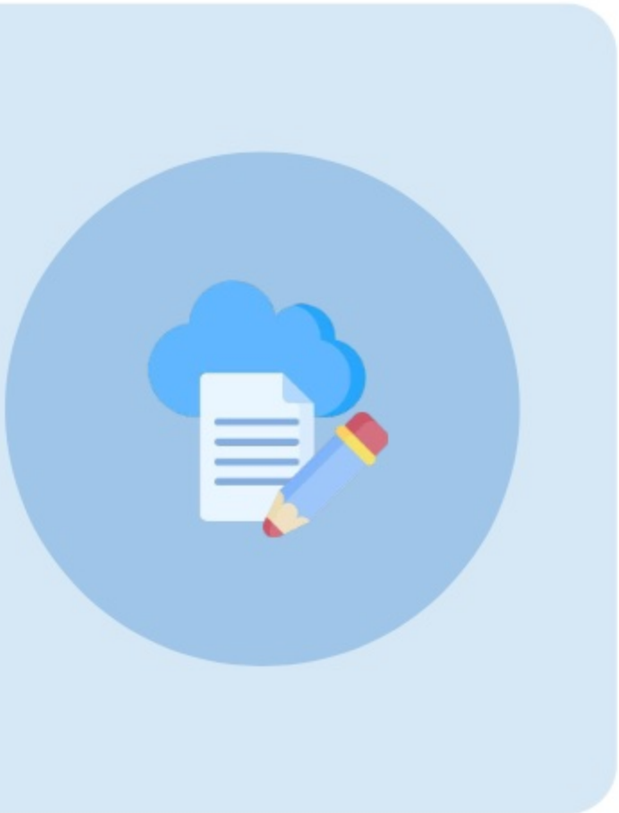
Write to Salesforce

The following describes **error handling** considerations when an error is encountered upon **writing to Salesforce**.

ERROR HANDLING

If an error is encountered when **writing to Salesforce**, the Salesforce API returns the following information, which can be used for **retrying the operation**:

- Data for identifying the record (record Id, etc.)
- Success or failure notification
- Collection of errors of each failed record



Write to Salesforce

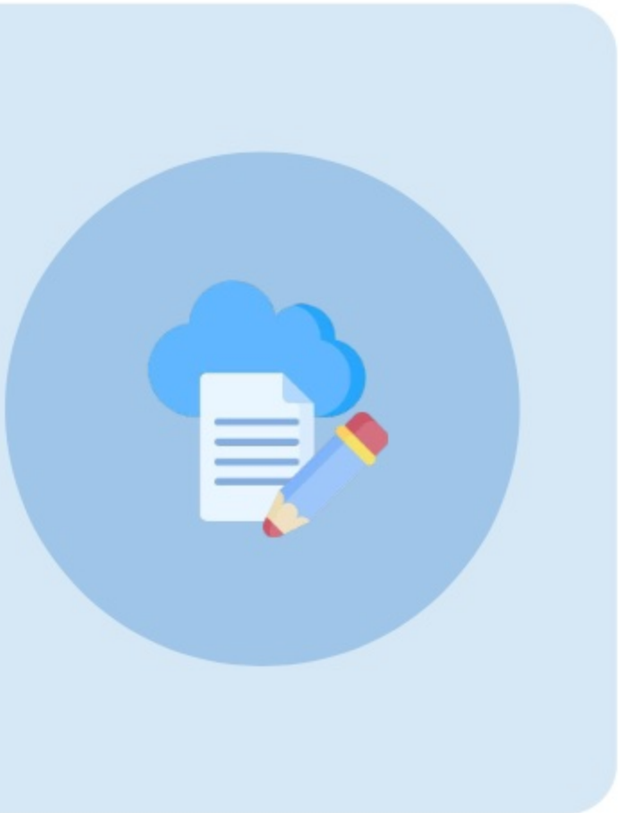
The following describes **error handling** considerations when an error is encountered upon **writing to Salesforce**.

RECOVERY

To recover from a **failed write operation**, the ETL process should be **restarted**.

FAILED RECORDS

If there are failed records even though the **write operation was successful**, an **immediate restart** of the operation should fix the issue. On the other hand, a **delayed restart** may be beneficial as it allows time for data to correct itself in Salesforce which may be causing the errors.



External Master System

The following describes **error handling** considerations when an error is encountered in an external master system.

ERROR HANDLING

In this case, the error handling and any internal recovery mechanism should be assumed and managed by the **remote system** and carried out in **accordance with their best practices** to ensure efficiency.

Remote Call-In

Table of Contents

Error Handling

In the **Remote Call-In** pattern, the **remote system** is the caller and is **required to handle errors** that may result from requests that are sent to Salesforce when using one of the **solutions** below.

SOAP API

The remote system uses standard SOAP API to read/write Salesforce data and other operations.

REST API

The remote system uses standard REST API to read/write Salesforce data and other operations.

APEX WEB SERVICES

Apex class methods can be exposed as web service methods to perform more custom and advanced logic.

APEX REST SERVICES

Apex class can be exposed as REST resources which can perform multiple updates in a single transaction.

BULK API

The remote system can use Bulk API 2.0 to perform data operation that involves more than 2,000 records.

Error Handling Considerations

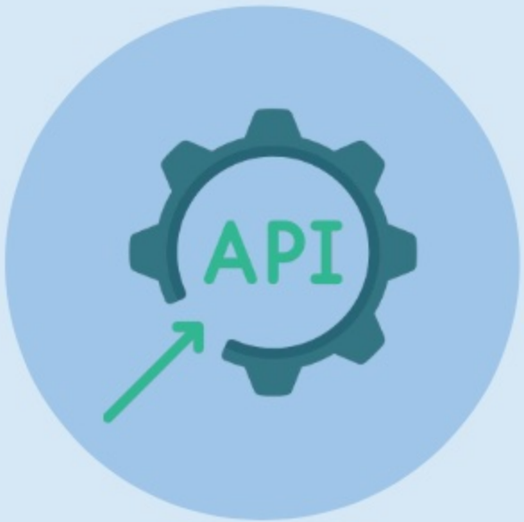
The following describes considerations in the **error handling** performed by the **remote system**.

TIMEOUTS AND RETRIES

When the remote system sends a request, it should implement a **timeout and retry mechanism** should Salesforce fail to respond.

MIDDLEWARE

A middleware can be used to act as a bridge between **Salesforce** and the **remote system** to provide logic for **error handling and recovery**.



Error Handling Considerations

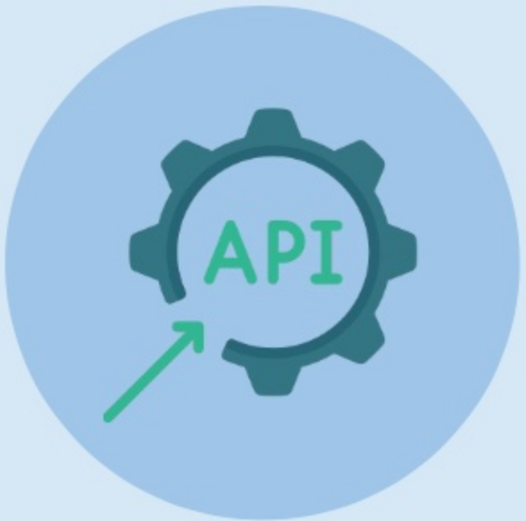
The following describes considerations in the **error handling** performed by the **remote system**.

PLATFORM EVENTS

The remote system can publish platform events by inserting a platform event object via SOAP/REST/BULK API. The **replay IDs** can be used in Salesforce to **fetch event messages** that are available within a certain period of time and allow responding to them accordingly.

IDEMPOTENT DESIGN

The remote system should be able to manage duplicate calls to **avoid duplicate inserts** and **redundant updates** in the Salesforce platform. If it is not equipped with idempotent capabilities, **repeated invocations** may result to **data integrity issues**.



Error Handling Considerations

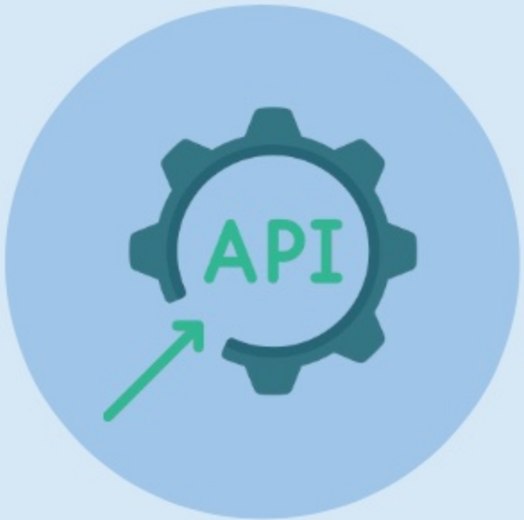
The following describes considerations in the **error handling** performed by the **remote system**.

SOAP EXCEPTION CODE

If errors occur due to **badly formed messages**, **failed authentication**, etc., a **SOAP fault message** with an associated **ExceptionCode** is returned by the SOAP API. The *ExceptionCode* can be used to identify the **specific cause** of the failure and handle the error accordingly.

SOAP ERROR DATA

When an error is encountered due to query-related issues such as a **create()** request, an Error data is returned by the SOAP API that contains the **statusCode**, **message**, **fields**, and **extendedErrorDetails** properties.



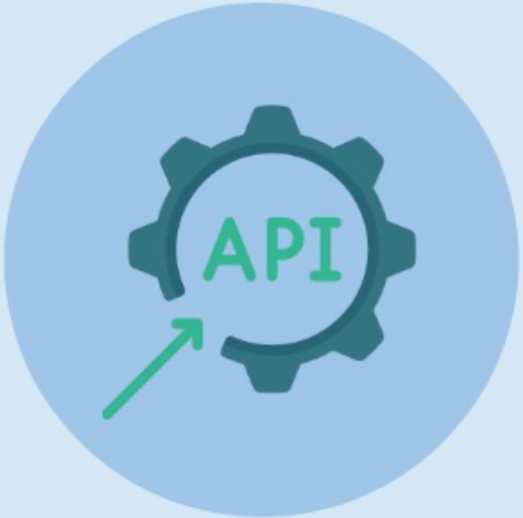
Error Handling Considerations

The following describes considerations in the **error handling** performed by the **remote system**.

SOAP SESSION EXPIRATION

When the **login()** call is used to sign on, a **new client session** is started. When the session expires after 120 minutes (default), an exception code **INVALID_SESSION_ID** is returned. To recover, login() can be called again.

Note that The SOAP API login will be **retired** in **Summer '27**. Salesforce recommends using **External Client Apps** and **OAuth** to authenticate external applications. The **OAuth Client Credentials Flow** or the **OAuth Web Server Flow** can be used depending on the use case.



Error Handling Considerations

The following describes considerations in the **error handling** performed by the **remote system**.

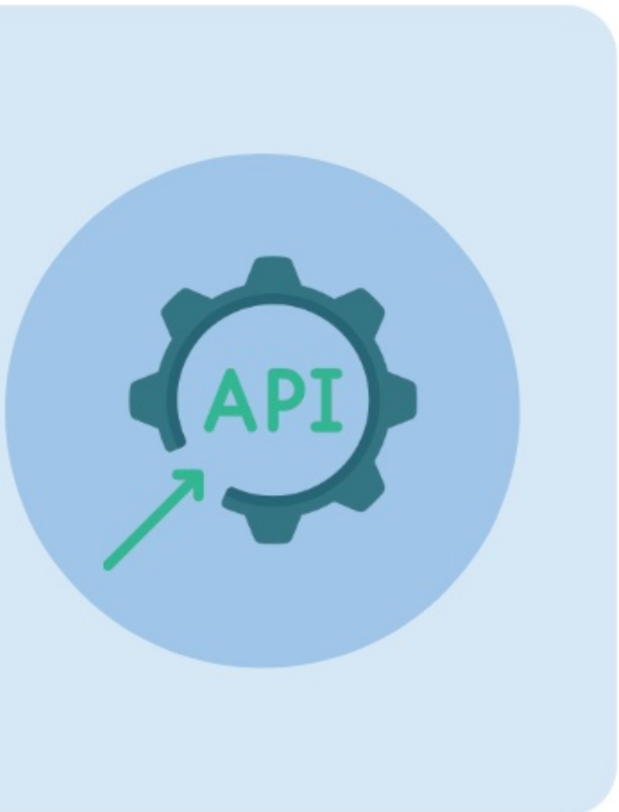
REST API RESPONSES

Whether an error is encountered or not, the **header** of a REST response contains an **HTTP code**. The **response body** usually contains:

- HTTP response code
- Message accompanied by the HTTP response code
- Field or object where the error occurred

HTTP RESPONSE CODE

Status codes that represent **failed requests** can be used to identify the specific cause of the failure and **handle the error** appropriately.



Error Handling Considerations

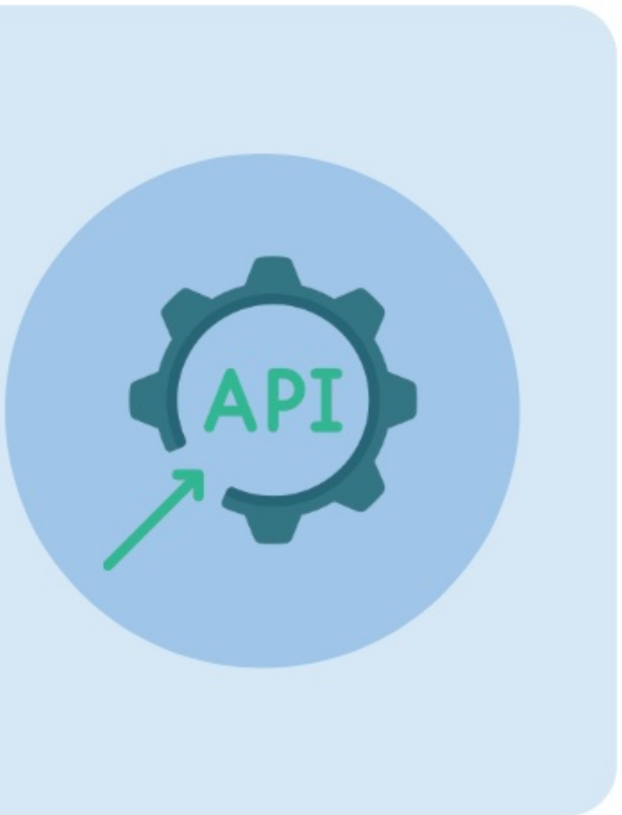
The following describes considerations in the **error handling** performed by the **remote system**.

BULK API ERRORS

Errors that occur when using Bulk API trigger **error codes**. How the error should be handled is based on the the **type of error** code that is triggered.

BULK API ERROR CODES

Some of the **most common** error codes encountered are **ExceededQuota** (job quota exceeded), **InvalidBatch** (invalid batch ID), **InvalidJob** (invalid Job ID), **InvalidSessionId** (invalid session ID), **Timeout** (connection timed out), **Unknown** (Unknown error occurred), etc.



Error Handling Considerations

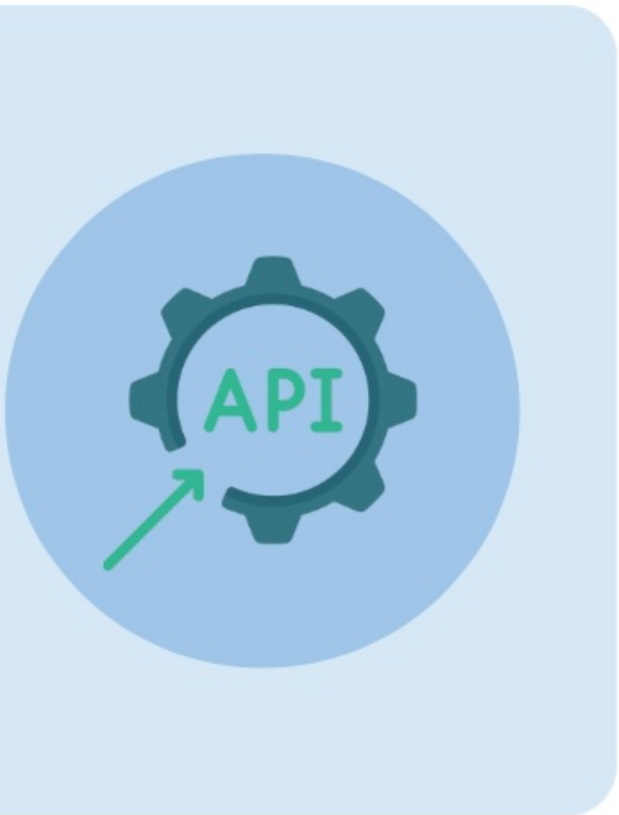
The following describes considerations in the **error handling** performed by the **remote system**.

BATCH FAILED RECORDS

After a BULK API operation has completed, the results of the operation can be retrieved by performing a GET request. In the returned batch result (XML, JSON, or CSV), the **Success** and **Error** properties can be used to identify which records failed in the transaction. Then, the failed records can be **fixed** if necessary and **submitted** again in a new batch.

BATCH TIMEOUT

When a **timeout error** is encountered when processing a batch, the batch can be split into smaller batches to minimize processing time for each batch.



UI Update
Based on Data Changes

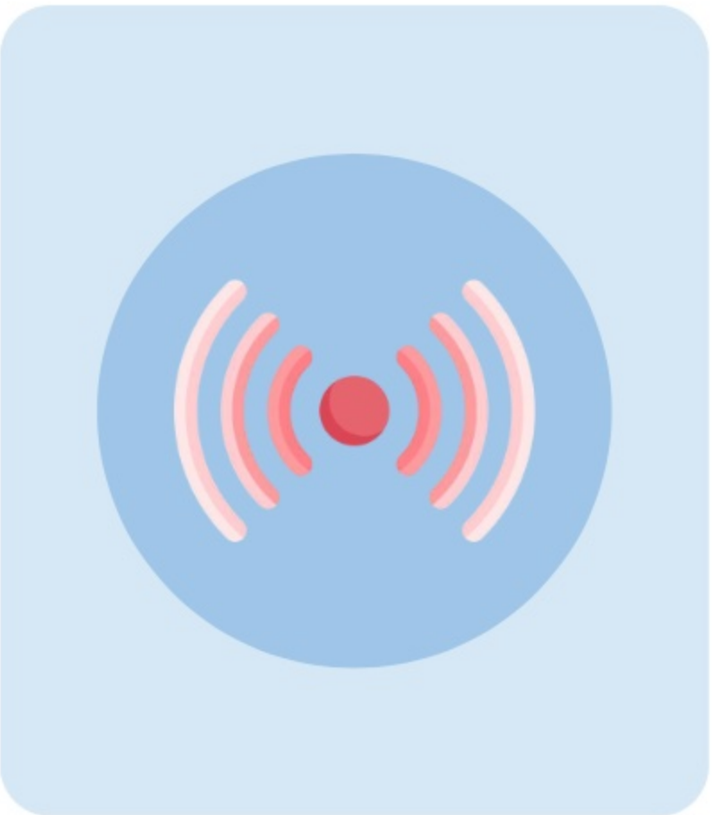
Table of Contents

Integration Solution

In this integration pattern, the **Streaming API** is used to publish events based on **Salesforce data changes** that allow channel subscribers to communicate the change to a **user interface**.

ERROR HANDLING

The streaming API is equipped with a **built-in error handling/retry mechanism**. The channel subscribers, such as a **Lightning component** or **Visualforce page**, should manage error handling when errors are returned from the **streaming channel**.



Error Handling

The following are some of the **common errors** returned by the **Streaming API**. How channel subscribers can **handle the errors** are also described.

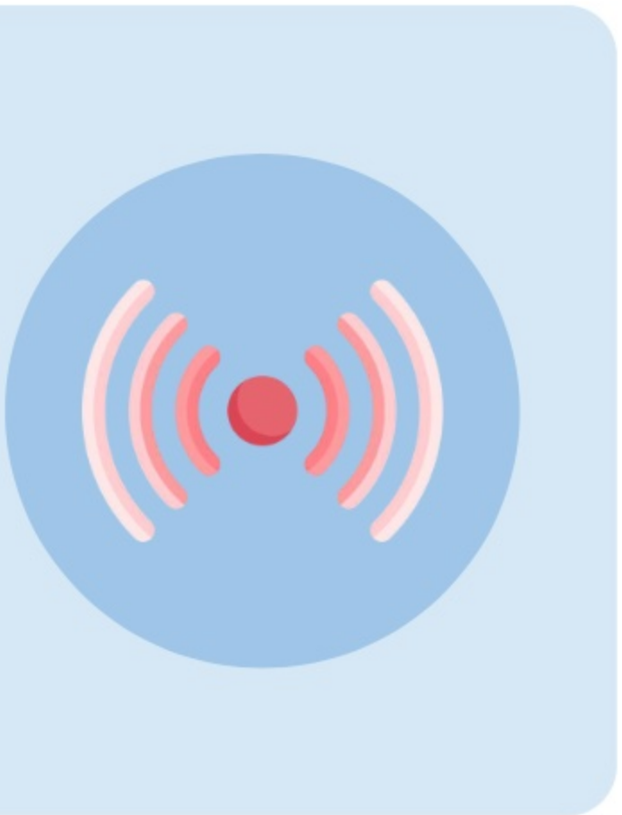
401 AUTHENTICATION ERROR

This error is encountered when **client authentication** is invalidated, for example, when the **OAuth token** or **Salesforce session** is revoked.

REAUTHENTICATE AND RECONNECT

If the following are met, then the client should reauthenticate and reconnect in order to recover and receive **new events**:

- The Bayeux (error) **message** is sent on the **/meta/connect** channel
- The **error** value is **401::Authentication invalid**
- The **advice** field contains **reconnect=none**



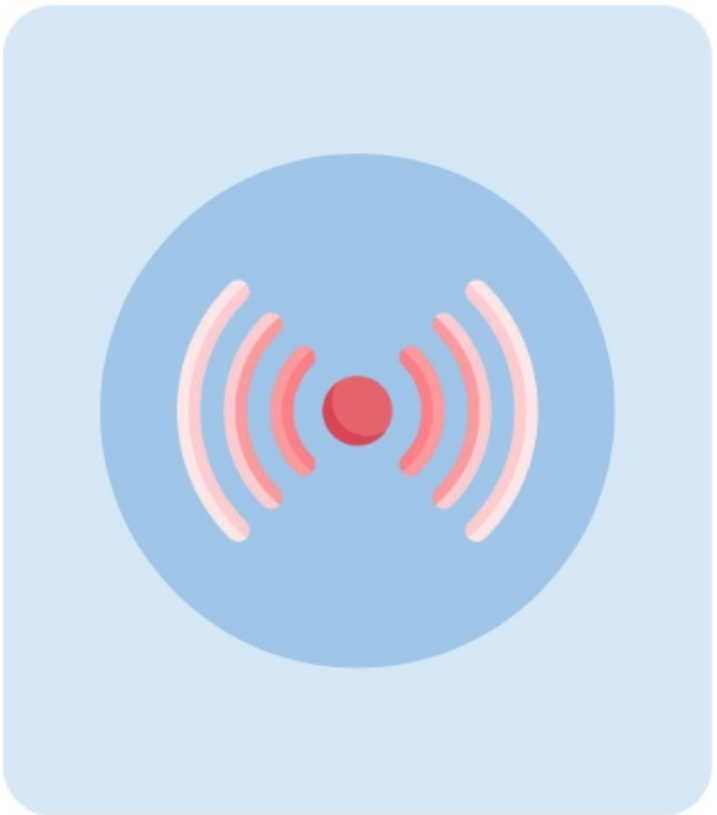
Error Handling

The following are some of the **common errors** returned by the **Streaming API**. How channel subscribers can **handle the errors** are also described.

PERFORM HANDSHAKE REQUEST

If the following are met, then the client should perform a new handshake request to recover from a **failed connection**:

- The Bayeux (error) **message** is sent on the **/meta/handshake** channel
- The **error** value is **403::Handshake denied**
- The **401::Authentication invalid** error is nested in an **ext** property



Error Handling

The following are some of the **common errors** returned by the **Streaming API**. How channel subscribers can **handle the errors** are also described.

403 UNKNOWN CLIENT ERROR

This error is encountered when a long-lived **connection failed** due to **network disruption**. In this case, the CometD server **times out** the client and **deletes the client state**.

ERROR HANDLING

If the following are met, then the client should **perform a new handshake**, and then if successful, should **resubscribe to the channel**:

- The **403::Unknown client** error is received in the error response
- The **advice** field in the response contains **"reconnect":"handshake"**

Error Handling

The following are some of the **common errors** returned by the **Streaming API**. How channel subscribers can **handle the errors** are also described.

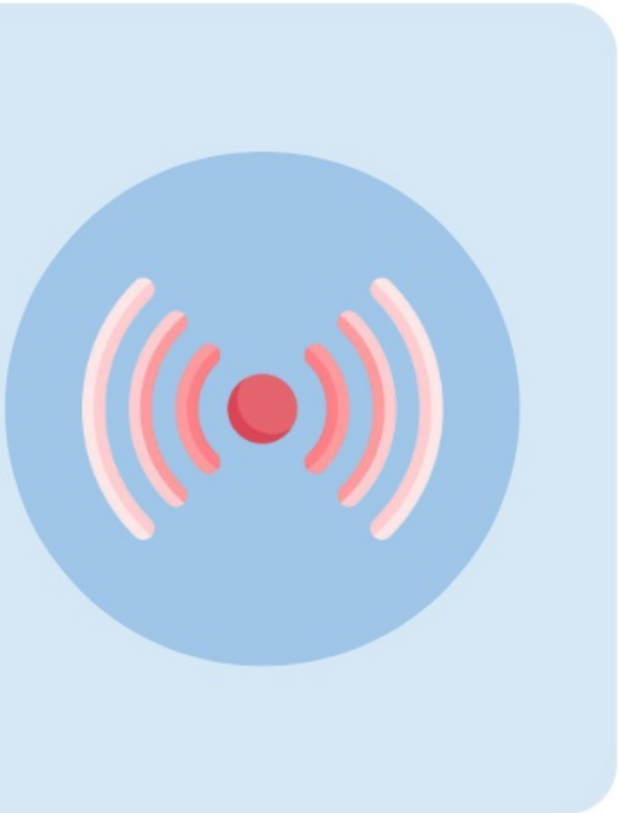
403 RESOURCE LIMIT AND VALIDATION ERRORS

This error is encountered when the client fails the validation during a **handshake request**. The Streaming API validates the client's **API version**, **maximum concurrent limits**, and **simultaneous connection limit**.

ERROR HANDLING

If the following are met, then the client should check its **API version** and/or avoid exhausting its **resource limits** in order connect successfully:

- The **403::Handshake denied** error is received in the error response
- The **failure reason** is specified and nested in an **ext** property



Error Handling

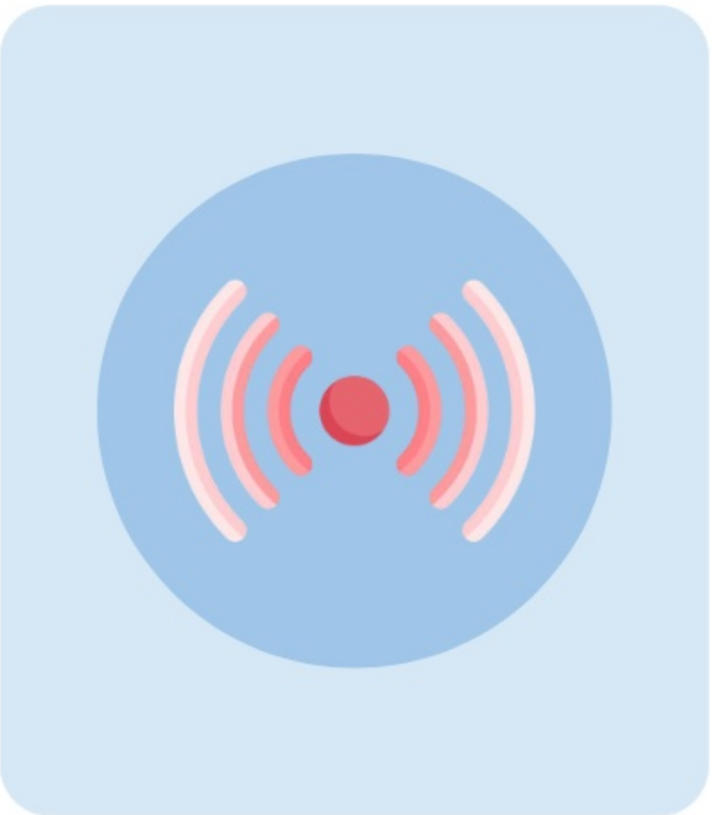
The following are some of the **common errors** returned by the **Streaming API**. How channel subscribers can **handle the errors** are also described.

503 SERVER TOO BUSY ERROR

This error is encountered when Salesforce **does not have available resources** to process the Streaming API request.

ERROR HANDLING

In this type of error, a **503::Server is too busy** is specified as the failure reason in the **ext** property of the error response. As this error is only **temporary**, retrying the request **after a few minutes** should fix the issue.



Data Virtualization

Table of Contents

Integration Solutions

In the **Data Virtualization** pattern, the following **integration solutions** can be used.

SALESFORCE CONNECT

Salesforce Connect provides the capability to access and work with data from external sources inside Salesforce.

REQUEST AND REPLY

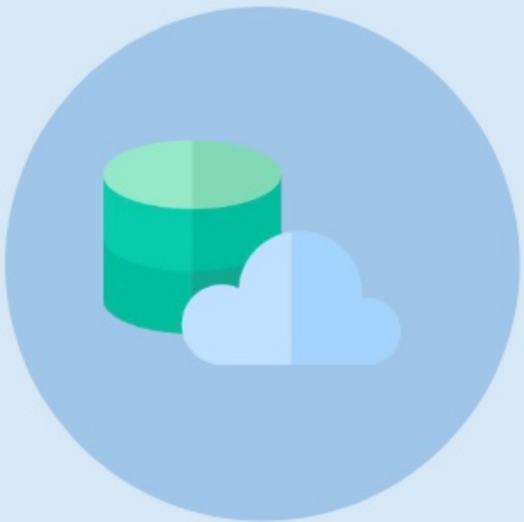
Lightning components or Visualforce pages can be used to perform SOAP/REST callouts to remote system.

Error Handling

In this integration pattern, when **exceptions or error codes** are returned from a request, the **caller** (Salesforce) should manage the error handling.

SALESFORCE CONNECT VALIDATOR

The Salesforce Connect Validator tool, which can be downloaded from **AppExchange**, can be used to identify any potential problems related to **external objects**. The tool runs common queries to test for **duplicate IDs**, **sorting**, **filtering**, **creating**, and **deleting** records. Also, the **error types** and **failure causes** of failed queries will be specified.



Error Handling

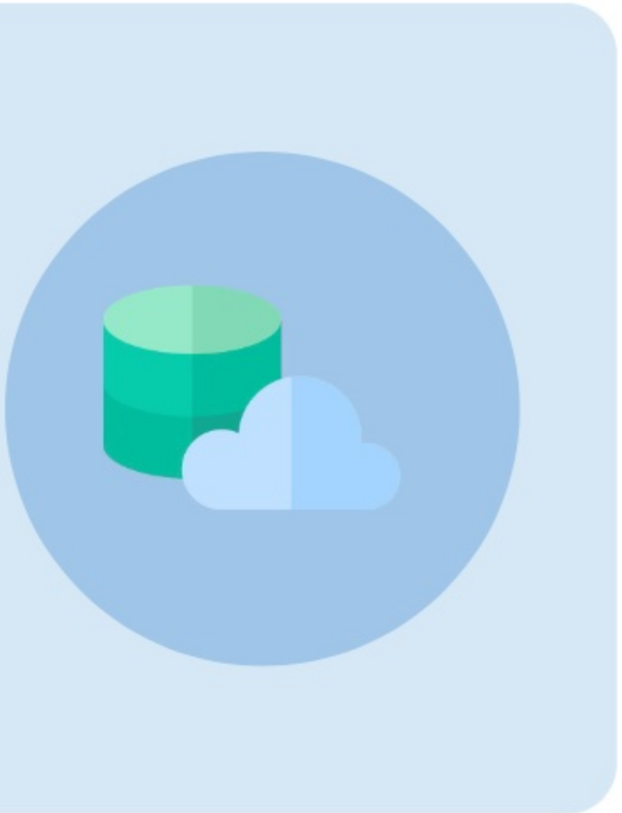
In this integration pattern, when **exceptions or error codes** are returned from a request, the **caller** (Salesforce) should manage the error handling.

WRITING TO EXTERNAL OBJECTS

When using Apex to write to **External Objects**, asynchronous Database commands such as **insertAsync()**, **updateAsync()**, and **deleteAsync()** are used. These methods work around potential issues caused by **network latency** and **external system availability**. Moreover, they support **callback functions** which contain results and status of the operation.

BACKGROUND OPERATION

When Apex is used to work with an **external object**, a job is queued and its status, progress, or any related errors can be viewed by accessing the BackgroundOperation object via the **API** or **SOQL**.



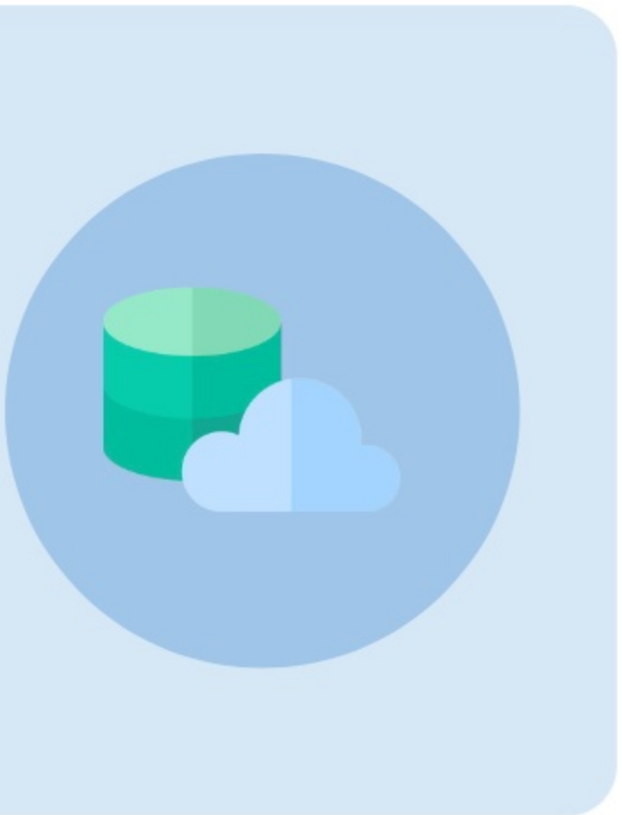
Error Handling

In this integration pattern, when **exceptions or error codes** are returned from a request, the **caller** (Salesforce) should manage the error handling.

RATE LIMIT ERRORS

When using **Salesforce Connect adapters** with imposed **callout limits** and rate limit errors are encountered, one or more of the following can be considered to avoid the error:

- Selecting **High Data Volume** in the external data source definition to bypass most rate limits (some behaviors and limitations are applied)
- Running fewer **SOQL and SOSL queries**
- Modifying Apex to cache **frequently accessed** data from the remote system that **seldom changes**
- Requesting a **higher rate limit** from Salesforce support

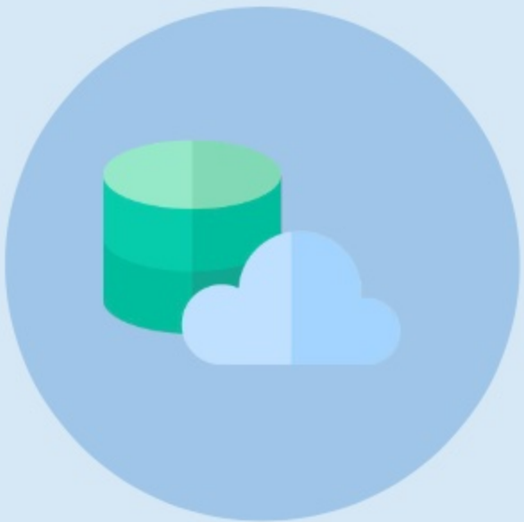


Error Handling

In this integration pattern, when **exceptions or error codes** are returned from a request, the **caller** (Salesforce) should manage the error handling.

MIXED DML ERRORS

A mixed DML error will be encountered in a scenario when a record-triggered flow, for example, creates an external object record after a Salesforce contact is updated. To avoid this **mixed transaction error**, the external object can be performed in a different transaction, for instance, using the **asynchronous path** in Flow Builder.

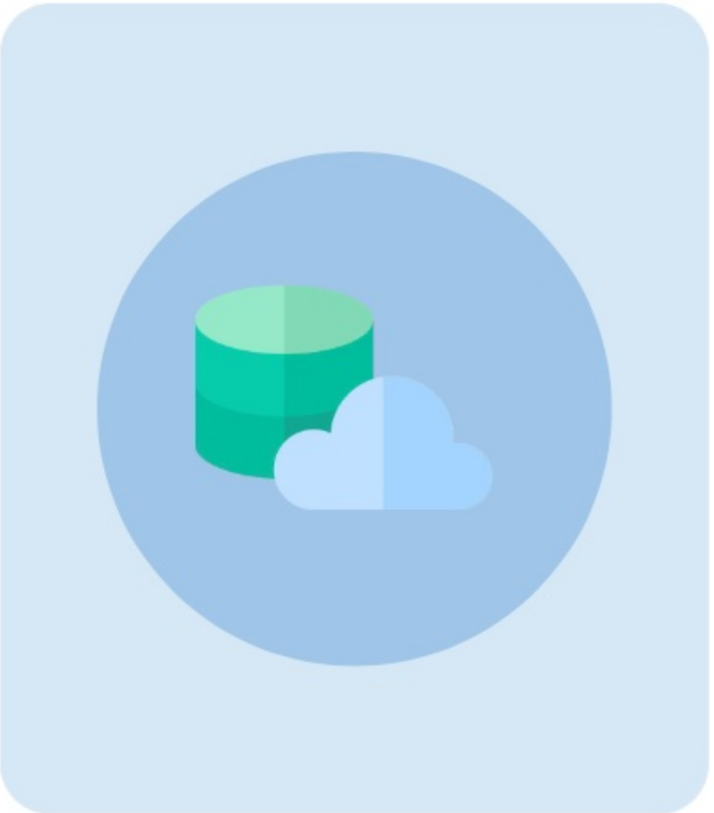


Error Handling

In this integration pattern, when **exceptions or error codes** are returned from a request, the **caller** (Salesforce) should manage the error handling.

SOAP API

A Visualforce page, for example, can be used to perform synchronous **Apex SOAP callouts** to the remote system. Salesforce should then handle the response accordingly, whether the request resulted in a success or failure. This solution follows the **Remote Process Invocation - Request and Reply** integration pattern.

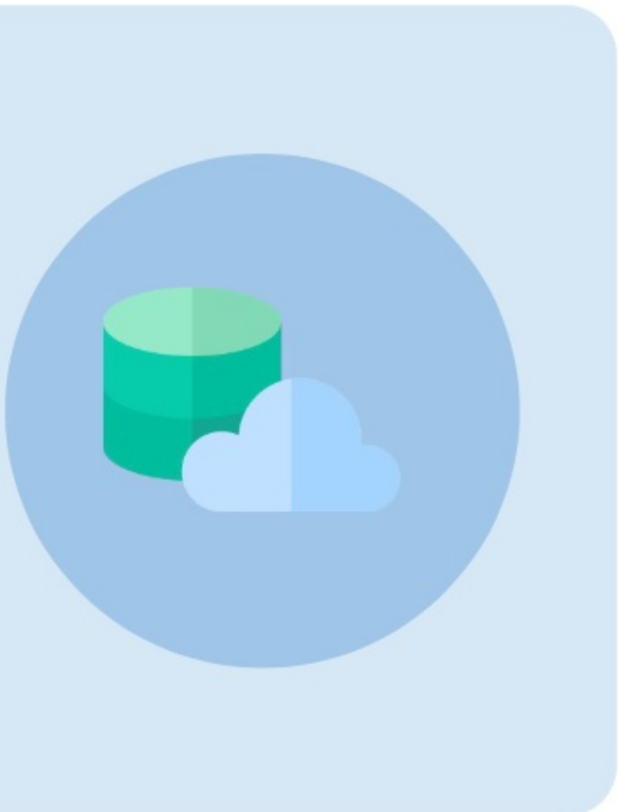


Error Handling

In this integration pattern, when **exceptions or error codes** are returned from a request, the **caller** (Salesforce) should manage the error handling.

REST API

A custom Lightning component, for example, can be used to perform synchronous **Apex HTTP callouts** to a REST service endpoint on the remote system. Salesforce should then handle the response accordingly, whether the request resulted in a success or failure. This solution follows the **Remote Process Invocation - Request and Reply** integration pattern.



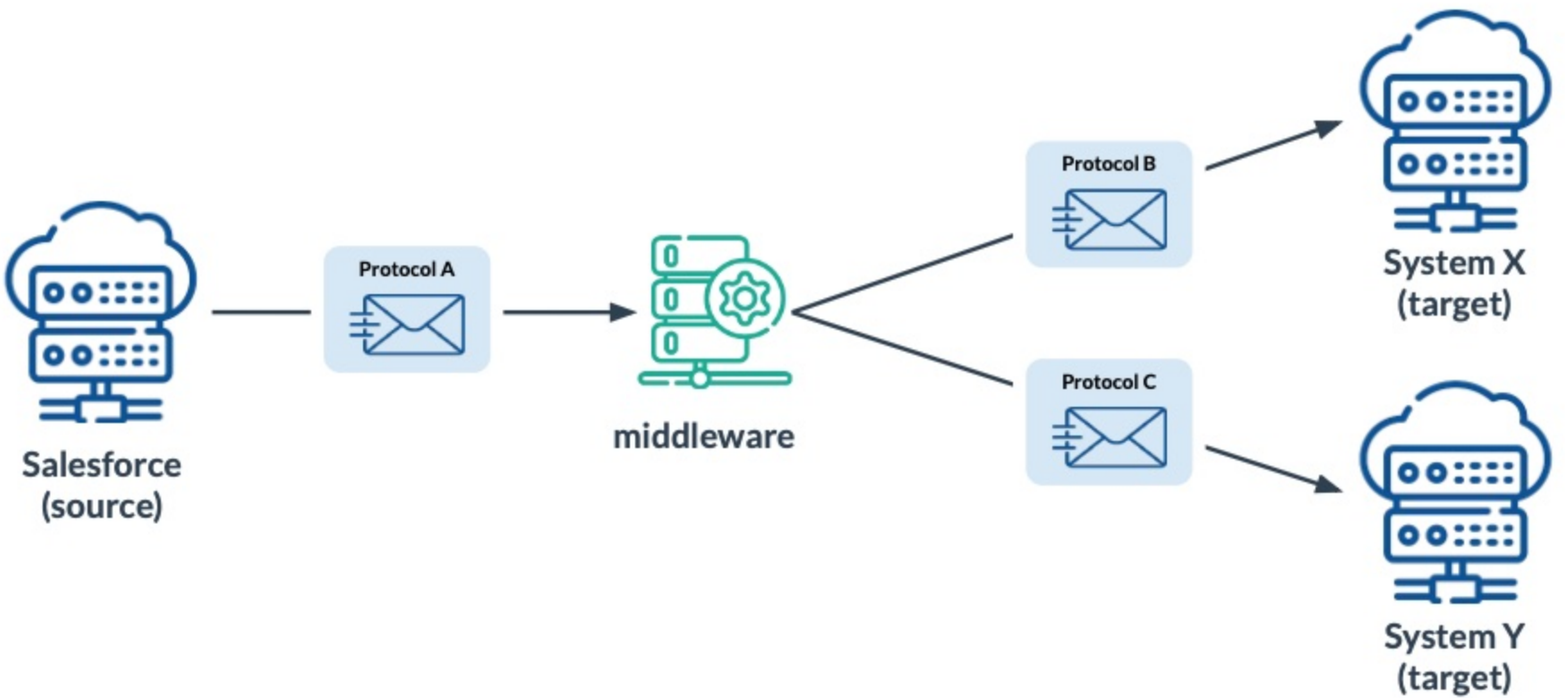
Middleware

Table of Contents

Middleware

A middleware is a tool that acts as a ”middleman” between one system and another to **allow communication** regardless of the native protocols and data format required by each platform.

A middleware is capable of handling **synchronous** or **asynchronous** events from a source and relays the event or handles the data distribution to the necessary endpoints or targets.



Middleware

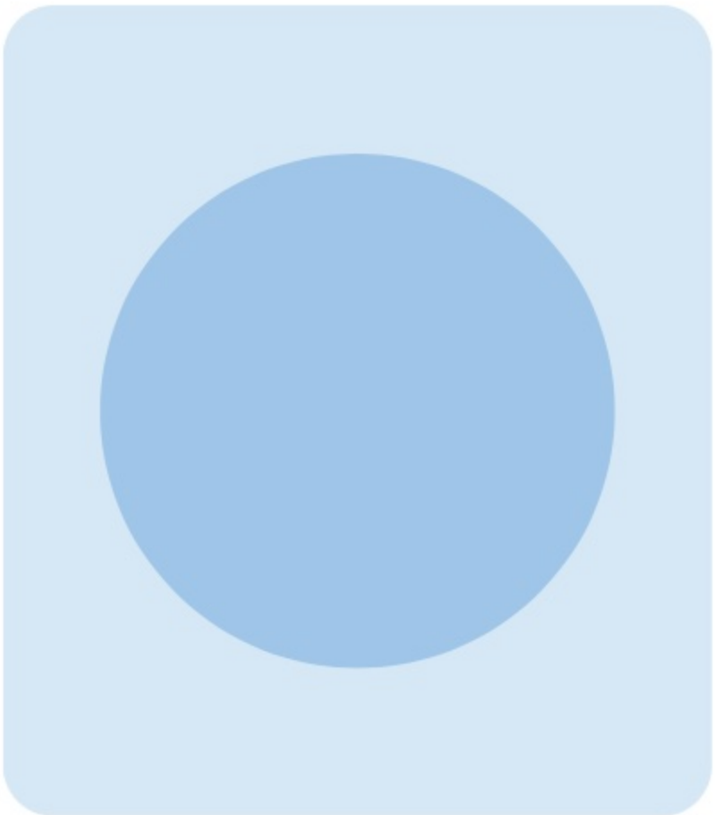
The following provides details on the **characteristics** and **capabilities** of a middleware.

EVENT HANDLING

Middleware identifies where an event should be forwarded and forwards it to the necessary recipient, or broadcasts the event in a publish/subscribe scenario. The middleware may perform other actions such as writing to a log, sending an error or recovery process, etc. In Salesforce, this capability can be achieved using platform events.

PROTOCOL CONVERSION

Native protocols in one system may not be compatible with the protocols of the other system and prevents connectivity. Middleware can convert the message format or encapsulate the payload using a format or protocol that is understood by the target system to establish connectivity.



Middleware

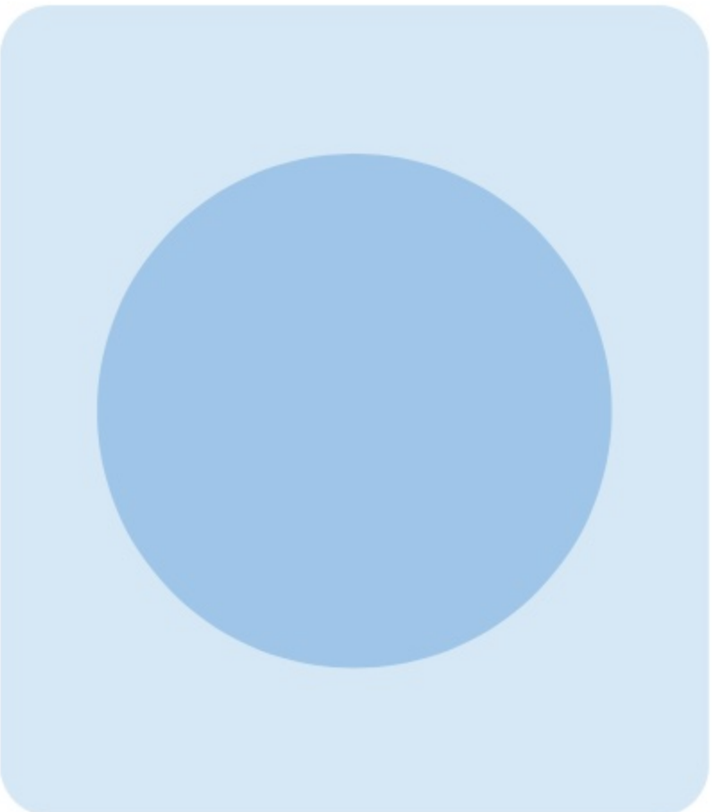
The following provides details on the **characteristics** and **capabilities** of a middleware.

TRANSLATE & TRANSFORM

Middleware can be used to translate and transform payload data so that the message can be readily mapped or consumed by the target system. Although Apex can be used to model the message structure for immediate processing by the target system, it is not recommended due to maintenance and performance considerations.

QUEUE & BUFFER

Middleware support asynchronous message processing where messages sent from the sender is stored temporarily by the middleware in the event where the target system is preoccupied or may not be available due to connectivity issues. The messages will be sent by the middleware once the target system is ready.



Middleware

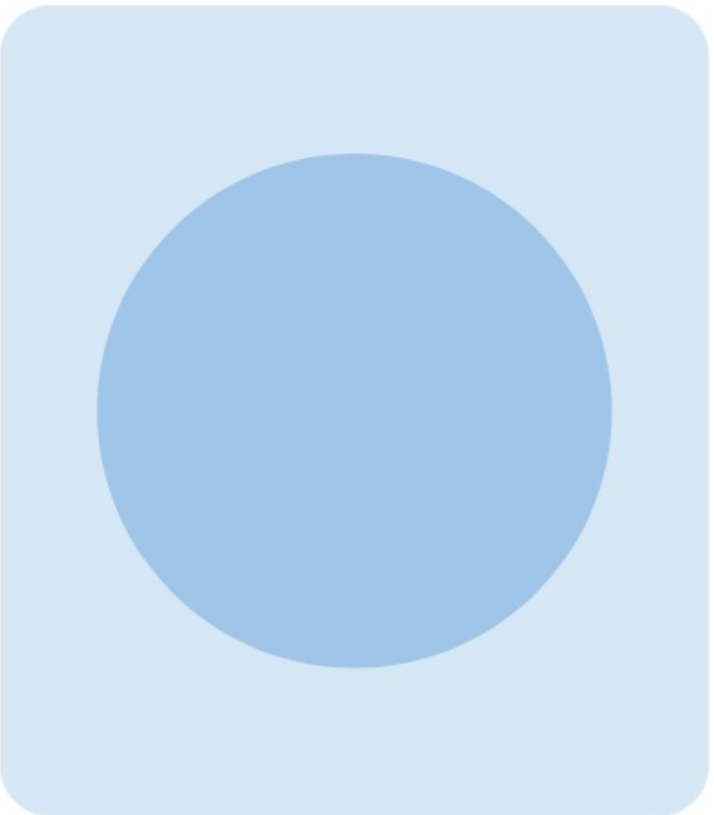
The following provides details on the **characteristics** and **capabilities** of a middleware.

TRANSPORT PROTOCOLS

Middleware supports synchronous and asynchronous transport protocols. In synchronous transport protocol, only one request is allowed to be processed in a single thread such as a request-and-response behavior. In asynchronous transport protocol, multiple requests are allowed in a single thread through the use of callbacks.

ROUTING

Instead of the sender directly talking to the recipient, the middleware forwards the message and ensures it is successfully received and understood by the designated recipient through a route amongst a range of available routes. Although Apex can be used to route messages, it is not recommended due to maintenance and performance considerations.



Middleware

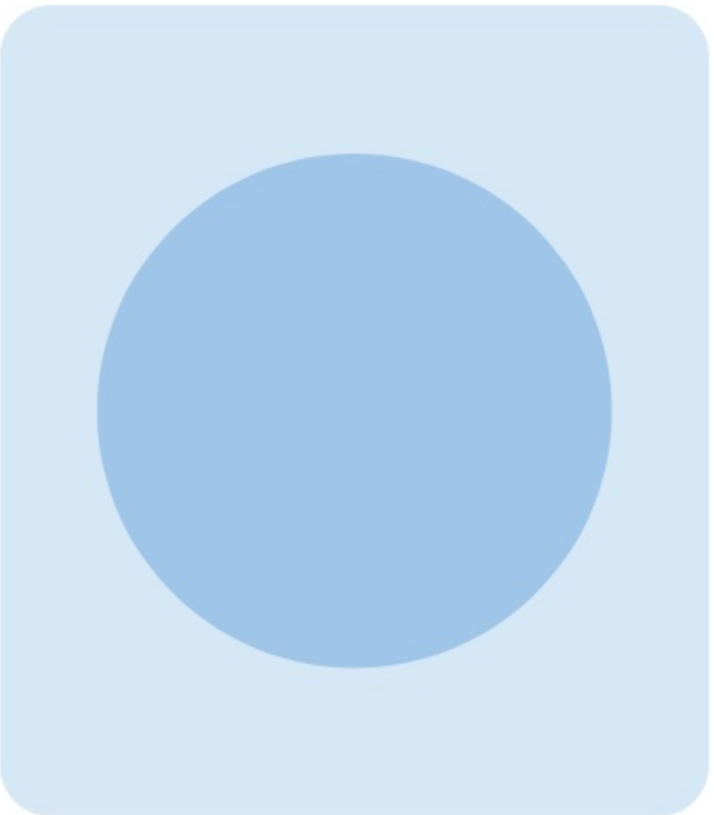
The following provides details on the **characteristics** and **capabilities** of a middleware.

PROCESSES & SERVICES

Services can be choreographed so that they operate independently yet coordinate with each other. Services can also be orchestrated where tasks are performed through coordination by a central conductor. While some choreographies can be built into Salesforce, complex orchestrations should be handled by the middleware due to Salesforce timeout values and governor limits.

TRANSACTIONALITY

Middleware supports all four ACID (atomicity, consistency, isolation, durability) properties. Salesforce recommends using a middleware for solutions that require complex, multi-system transactions since it is not capable of participating in transactions that occur externally.



Middleware

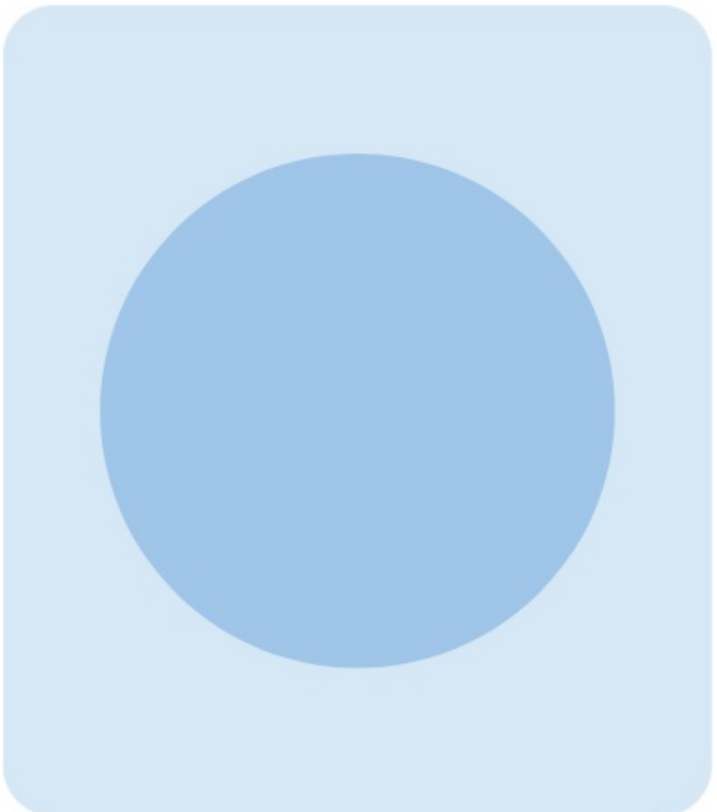
The following provides details on the **characteristics** and **capabilities** of a middleware.

EXTRACT, TRANSFORM, AND LOAD

A middleware extracts data from a source system, transforms the data for compatibility and quality, and loads the data to the target system. (Extract, Transform, and Load) ETL tools may support a change data capture functionality, like the Change Data Capture feature in Salesforce.

LONG POLLING

Middleware supports long polling (also called *Comet* programming). If data requested by the client is not yet available, the server holds the request and maintains the connection until the data is available (instead of already responding to the client). When the information is finally available, the server completes the request by sending the data back to the client. In Salesforce, the Bayeaux protocol and CometD is used by the Streaming API to implement long polling.



Use Cases

Table of Contents

Scenario 1

SCENARIO

A program and events company syncs their account records from a sales partner that uses **Change Data Capture** in Salesforce. The company is able to consume the **change events** over a **CometD** connection. At certain times though, a **gap event** would be received by the company instead of the required change event.

Solution 1

SOLUTION

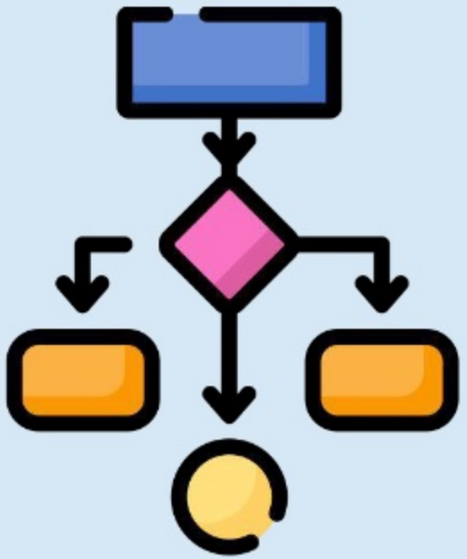
Change Data Capture publishes a **gap event** when it is unable to publish a **change event** due to internal issues or certain processes that are in progress in Salesforce at that time. The gap event includes **record IDs** of the changed records and other information in the event payload.

So, if a gap event is received, the **consumer** should retrieve the updates by initiating a request to Salesforce to fetch record data identified by the **record IDs**.

Scenario 2

SCENARIO

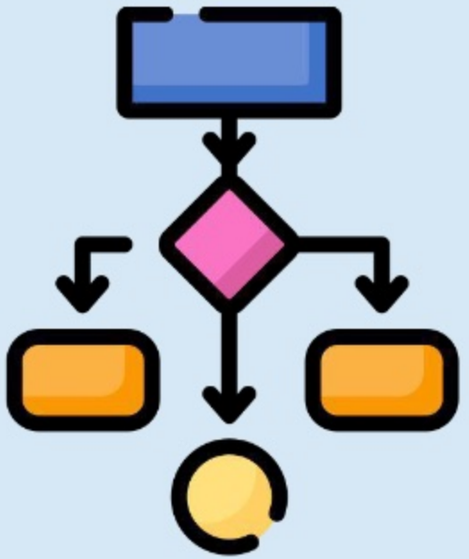
External Services is used to manage the pension scheme for all employees of a real estate company that uses Salesforce. A **schema definition** is specified with status codes to allow **Flow Builder** to handle successful and failed transactions when Apex actions are executed. However, in some occasions, the remote system fails to respond and causes **unhandled errors** in Flow Builder.



Solution 2

SOLUTION

The **status codes** in a schema definition allow Flow Builder to execute the next path accordingly based on the returned status. However, if the **remote system** fails or other unforeseen issues are encountered which causes **unhandled errors** in Flow Builder, a **fault path** can be used on the **Apex action** to execute a path that handles the error properly. For example, the fault path can either send a custom email or display a screen to the user.



Scenario 3

SCENARIO

A vehicle testing headquarter uses Salesforce to manage several smoke emission testing centers that caters to different types of vehicles region-wide. When vehicles pass or fail tests, events are published through a **Streaming API channel** to notify users of a subscribed **Lightning component** regarding the vehicle status. In multiple occasions, notifications are not received even if tests have been completed and submitted. Upon checking the logs, **403::Handshake denied** errors can be seen.

Solution 3

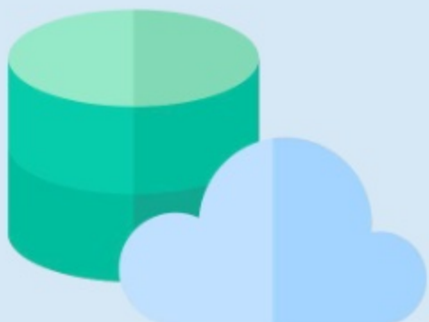
SOLUTION

To handle errors in a solution that uses the Streaming API, the error can be identified by checking the **error code** and/or **failureReason** properties of the response. In this case, when the error code is **403::Handshake denied**, the failure Reason, for example, may indicate that the number of simultaneous connections per server **has exceeded the limit**. If so, either the number of clients allowed to connect to the streaming channel **should be minimized**, or the streaming client can **try again** in a later request.

Scenario 4

SCENARIO

An enterprise reseller uses **Salesforce Connect** to allow its sales team to remotely access from a **Visualforce page** the products of their partner that manufactures modern furnitures for schools and offices. The team uses the tool to also **build reports**, and **query orders** and **customer information** in the remote system. During peak hours, users may encounter **rate limit errors** when trying to access data that interrupt work and affect productivity.



Solution 4

SOLUTION

Salesforce Connect performs **callouts** when accessing data from the remote system. These callouts, depending on the **type of adapter** used, may be **governed by limits** that throw errors when exceeded. In this case, the data volume is **not high**. So, instead of minimizing SOQL/SOSL requests as this would affect productivity, Apex can **cache data** that seldom changes to **reduce the number** of callouts. If errors are still encountered, a **higher rate limit** can be request from Salesforce support.

It is important to note that while there are limits imposed on **custom** and **cross-org adapters**, there are **no callout limits** for **OData** and **other adapters** such as Amazon DynamoDB adapter, GraphQL adapter, etc.

Learn More

- Integration Patterns and Practices
- Use External Services With a Flow
- Error Handling Best Practices for Lightning Web Components
- Error Handling Best Practices for Lightning and Apex
- SOAP API - Error Handling

Learn More

- REST API - Status Codes and Error Responses
- BULK API - Errors
- Change Data Capture - Gap Events
- Handling Streaming API Errors
- Salesforce Connect